JAVA INTERVIEW PREPARATION

CHAPTER 18

CONCURRENTHASHMAP INTERNALS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

ConcurrentHashMap Internals: Atomic Updates Without a Global Map Lock

Senior / Lead Java Developer Reading Chapter

ConcurrentHashMap supports high-concurrency access while preserving defined atomic operations. It is not merely a HashMap with synchronized methods. Its design allows retrievals to proceed with very little coordination, updates to contend mainly around affected buckets, and resizing work to be shared. The class solves data-structure synchronization, but it cannot automatically make a multi-step business workflow atomic.

Why a Synchronized HashMap Is Different

Wrapping a map with Collections.synchronizedMap places a common synchronization boundary around its methods. This can be correct, but unrelated keys contend for the same lock and compound iteration still requires the documented external synchronization.

```
Map<String, Session> sessions =
    Collections.synchronizedMap(new HashMap<>());

synchronized (sessions) {
    for (Session session : sessions.values()) {
        inspect(session);
    }
}
```

ConcurrentHashMap is designed for concurrent operations rather than global mutual exclusion. Retrievals generally do not acquire bin locks. Updates coordinate using compare-and-set operations and small synchronization regions associated with bins or table-management state.

Conceptual Structure in Modern JDKs

Modern implementations use a bucket table containing nodes. A bucket may be empty, contain a list, contain a tree, or temporarily contain a forwarding marker during resize.

```
table
 [0] -> node -> node
 [1] -> empty
 [2] -> tree bin
 [3] -> forwarding node ----> next table during resize
```

Older Java versions used a visible segment architecture. Describing current ConcurrentHashMap as a fixed collection of independently locked segments is outdated. Constructors may retain a concurrencyLevel parameter for compatibility and sizing guidance, but modern locking is not the old segment design.

Reads, Visibility, and Happens-Before

A successful retrieval of a value for a key reflects a happens-before relationship with the completed insertion or update that established that value. Internal volatile reads and publication rules allow readers to observe safely published nodes and values.

```
configuration.put("pricing", fullyInitializedConfig);

// A thread that later observes this mapping also observes the
// initialization actions that happened before the completed put.
Configuration config = configuration.get("pricing");
```

This guarantee applies to the map publication action. If the value is mutable and is later changed without synchronization, ConcurrentHashMap does not make those later fields magically thread-safe.

Update Mechanics

For an empty bucket, insertion can often install a node with compare-and-set. For a populated bucket, an updater coordinates on that bin while locating or appending the entry. Therefore two threads updating different buckets can often proceed independently.

```
Thread A -> bucket 2 ---- lock/coordinate here
Thread B -> bucket 9 ---- lock/coordinate independently
Thread C -> bucket 2 ---- may contend with A
```

Collision quality still matters. Many keys concentrated into one bin reduce concurrency as well as lookup performance. Tree bins control lookup degradation but cannot eliminate contention on a hot logical key.

Atomic Map Operations

The most important API feature is not internal locking; it is atomic compound operations.

```
cache.putIfAbsent(key, createValue());
cache.computeIfAbsent(key, this::loadValue);
cache.computeIfPresent(key, (k, old) -> refresh(old));
cache.compute(key, (k, old) -> calculate(k, old));
cache.merge(key, delta, Integer::sum);
cache.replace(key, expectedValue, newValue);
cache.remove(key, expectedValue);
```

These express intent inside the map's synchronization boundary. A manual containsKey followed by put has a race: multiple threads can all observe absence and create competing values.

computeIfAbsent Is Not a General Transaction

The mapping function should be short, deterministic enough for the application, and free of recursive updates to the same map that can violate the operation's rules. It must not return null when a mapping is required.

```
Customer customer = customers.computeIfAbsent(
    customerId,
    id -> repository.load(id)
);
```

This prevents competing successful installations for the same key, but it does not turn the repository call into a distributed transaction. Slow I/O in mapping functions can create contention for callers interested in the same mapping. Failure, retry, stale data, eviction, timeout, and cancellation still require a cache policy.

For one `computeIfAbsent` invocation, Java 21 specifies that the function is invoked exactly once if the key is absent, or not at all when it is present. That is not an exactly-once business guarantee across time: a failed computation installs nothing, a caller may retry, and removal can allow a later invocation to compute again. Keep externally visible side effects idempotent or separate them from map population.

Nulls Are Deliberately Forbidden

ConcurrentHashMap rejects null keys and values. In a concurrent map, null from get must unambiguously mean that no mapping was observed. Allowing stored null values would make absence and presence difficult to distinguish during concurrent changes.

```
Value value = map.get(key);
if (value == null) {
    // no mapping was observed at this point
}
```

This is an observation, not a permanent fact. Another thread may add the key immediately afterward.

Size Is a Moving Observation

Under concurrent mutation, size, isEmpty, and containsValue describe a transient aggregate. They are useful for monitoring and approximate decisions, not for establishing a global invariant.

```
if (map.size() < limit) {
    map.put(key, value); // does not atomically enforce the limit
}
```

Use a Semaphore, bounded cache, explicit lock, database constraint, or another structure designed to enforce the actual limit. A concurrent collection makes individual operations safe; it does not freeze the world between them.

Cooperative Resizing

The table grows when population and bin distribution require more capacity. Resizing is expensive, so modern ConcurrentHashMap allows multiple updating threads to help transfer bins to the new table. Forwarding nodes tell readers and writers that a bin has moved.

```
old table                 new table
 bucket 4 --moved-------> buckets 4 and 20
 bucket 5 --worker A----> ...
 bucket 6 --worker B----> ...
```

Correct initial sizing still matters for very large maps. It reduces transfer work and temporary memory pressure, though premature massive sizing wastes memory and iteration locality.

Weakly Consistent Iteration

Iterators do not throw ConcurrentModificationException merely because another thread updates the map. They are weakly consistent: they can reflect some modifications during traversal, never require a single global snapshot, and do not return structurally corrupt data.

This property supports monitoring and maintenance operations that tolerate an evolving view. It is not suitable when a report requires an exact point-in-time state. For that, create a snapshot under an application-defined consistency boundary or read from a versioned source.

Counters with LongAdder

A common pattern stores LongAdder values for frequently updated statistics.

```
ConcurrentHashMap<String, LongAdder> counts = new ConcurrentHashMap<>();

counts.computeIfAbsent(eventType, ignored -> new LongAdder())
      .increment();
```

LongAdder spreads highly contended updates across internal cells and combines them when sum is requested. Its sum is not an atomic transaction with concurrent increments, so it suits telemetry and frequency maps better than financial balances or strict limits.

Per-Key Atomicity Versus Cross-Key Invariants

Atomic methods protect operations associated with the relevant mapping. They do not atomically coordinate an invariant across multiple keys.

```
inventory.compute(productA, reserveOne);
inventory.compute(productB, reserveOne);
```

If both reservations must succeed or neither may succeed, two compute calls are insufficient. Use a higher-level lock with stable ordering, a transaction in the authoritative database, an actor/partition model, or redesign the aggregate boundary.

Similarly, storing a mutable ArrayList as a value is unsafe unless access to that list is separately controlled. The map safely publishes and locates the reference, not every operation on the referenced object.

Bulk Operations

ConcurrentHashMap provides parallel-capable forEach, search, and reduce operations with a parallelism threshold. These operations observe a concurrently changing map and require side-effect-safe functions.

Parallel execution is not automatically faster. Map size, function cost, common-pool contention, CPU availability, and latency requirements determine whether it helps. Use a high threshold or sequential operation for small maps and measure realistic workloads.

Initialization and the First Insertion

ConcurrentHashMap can defer allocation of its main table until it is first needed. Initialization itself must be coordinated so multiple threads do not create competing tables. Internal control fields encode states such as uninitialized, initializing, resize threshold, or resize in progress.

The important engineering lesson is that constructor parameters are sizing guidance, not a reservation of independent locks. The legacy `concurrencyLevel` argument does not mean that a modern map contains exactly that many segments. Treat it as historical API compatibility and avoid explaining current behavior through a fixed-segment diagram.

```
new map
  table = not allocated
          |
          | first update wins initialization coordination
          v
allocated power-of-two table
          |
          +-- empty bin: CAS node into slot
          +-- occupied bin: coordinate at that bin
```

Pre-sizing can reduce initialization and resizing work for known high cardinality, but enormous speculative capacity increases heap pressure. The initial-capacity argument describes an expected sizing input; it is not a hard maximum and does not enforce a business limit.

Read Paths and Publication Boundaries

A read computes a spread hash, locates a bin, and examines the bin structure. Ordinary retrievals generally do not lock and may overlap with writers. If a reader returns a non-null value established by a completed update for that key, the update happens-before that retrieval.

```
Thread A                              Thread B
initialize Value fields
map.put(key, value)  --publication-->  value = map.get(key)
                                         sees initialization before put
```

The edge does not freeze the object. If Thread A later changes a mutable field without a lock, volatile field, atomic variable, or another defined synchronization action, Thread B has no new visibility guarantee merely because the reference lives in ConcurrentHashMap. Prefer immutable values or give the value object an explicit concurrency design.

The guarantee is also per mapping, not a global snapshot. Reading key A and then key B can observe states produced by different moments in concurrent execution.

Bin Coordination and Hot-Key Contention

An empty-bin insertion can often succeed through compare-and-set. Once a bin is populated, updates involving that bin may synchronize on its first node or use specialized tree-bin coordination. This is localized compared with a single global lock, but localized does not mean contention-free.

```
well-distributed keys               hot/colliding keys
T1 -> bin 2                         T1 --+
T2 -> bin 7                         T2 --+--> bin 2 coordination
T3 -> bin 11                        T3 --+
updates overlap                    updates serialize or retry
```

A single popular key is inherently a serialization point when every update must transform its value atomically. Hash spreading cannot distribute one logical key over multiple bins. For metrics, sharded counters such as `LongAdder` reduce contention. For business state, consider partitioning by aggregate, batching, actor ownership, or moving the invariant to a transactional system.

Cooperative Resize in More Detail

During growth, the old and new tables can coexist temporarily. A forwarding node marks a migrated bin and directs operations toward the next table. Updating threads that encounter resize state may help transfer a range of bins instead of waiting for one dedicated resizer.

```
old table                                 next table
[0] normal nodes                          [0] transferred low side
[1] FORWARDING -------------------------> [1] / [1 + oldCap]
[2] claimed by worker A                   [2] / [2 + oldCap]
[3] claimed by worker B                   [3] / [3 + oldCap]

transfer complete -> next table becomes current table
```

This spreads resize work across mutators but does not make growth free. For a period, both arrays and transfer metadata are live, increasing allocation and memory pressure. A large resize during traffic can consume CPU and extend update latency. Diagnose cardinality growth and resize behavior rather than assuming every blocked update is ordinary bin contention.

Counter Striping and Approximate Aggregates

Maintaining one globally contended integer for the map's size would limit scalability. Modern implementations use a base count plus striped counter cells under contention, conceptually similar to `LongAdder`. Aggregate methods combine these moving counters.

```
update threads:   T1 -> cell0
                  T2 -> cell3
                  T3 -> baseCount
size estimate:    base + sum(cells)
```

That design explains why `size()` during active updates is an observation rather than a transaction boundary. Even a perfectly measured count could become stale immediately after it is returned. Enforcing “at most 10,000 active sessions” needs admission control or an authoritative atomic invariant, not `if (map.size() < 10_000)`.

Atomic Methods: Scope and Blocking Risks

Atomic remapping methods serialize the relevant mapping transition, but other attempted updates can be delayed while a computation is running. Mapping functions should therefore be short and simple. Remote I/O, database access, nested callbacks, and unbounded waits can turn a convenient method into hot-bin queuing.

```
// Risky when load performs slow or failure-prone remote I/O
Value value = cache.computeIfAbsent(key, this::loadRemotely);
```

For expensive loading, consider storing a future or a dedicated cache abstraction that defines expiry, timeout, cancellation, refresh, maximum size, and failure caching. Be careful with a future-valued map: a permanently failed future can poison the key unless removal and retry policy are explicit.

Recursive updates are particularly dangerous. Java 21 specifies an `IllegalStateException` when a detectably recursive computation would otherwise never complete. Do not rely on detection to make arbitrary re-entrancy safe; keep remapping functions focused on deriving the new value.

Views, Iterators, and Snapshot Requirements

`keySet`, `values`, and `entrySet` are backed views. Their iterators are weakly consistent: they can observe elements that existed at some point during traversal and do not throw `ConcurrentModificationException` merely because writers continue.

```
iterator starts:       {A, B, C}
concurrent changes:    remove B, add D
possible traversal:    A, C, D     or A, B, C     or another valid evolving view
not promised:          one atomic snapshot timestamp
```

Weak consistency is excellent for diagnostics, registries, and maintenance tasks that tolerate an evolving view. For billing, reconciliation, or a report requiring a precise cut, establish an application consistency boundary: copy while updates are stopped, read a database snapshot, use immutable versioned state, or process an event-log position.

ConcurrentHashMap Is Not Always the Answer

Use ConcurrentHashMap for concurrent keyed access and per-key atomic transitions. Choose another design when the invariant differs:

```
requirement                              stronger fit
-------------------------------------------------------------------
exact multi-key transaction               database / explicit lock
bounded expiring cache                    mature cache library
one owner processes ordered commands      actor or partition loop
sorted concurrent navigation              ConcurrentSkipListMap
immutable read-mostly configuration        atomic immutable snapshot
global lock around a small map invariant   synchronized map + lock discipline
```

One global lock can be the clearer correct design when operations must atomically inspect the entire map and contention is low. Sophisticated internals cannot remove a business invariant that is genuinely global.

A Strong Interview Explanation

Explain that modern ConcurrentHashMap is a bucket table using volatile publication, CAS for favorable insertions, localized bin coordination for updates, tree bins for heavy collisions, striped counts, and cooperative resizing. Do not describe it as the old fixed-segment implementation. Retrievals generally do not block, and a completed update for a key happens-before a non-null retrieval that observes it.

Then shift from implementation to contract. Atomic methods protect a mapping transition, not an arbitrary multi-key workflow. Iteration and size are moving observations, nulls are forbidden to preserve absence semantics, and mutable values need their own synchronization. Conclude with production risks: hot keys, slow mapping functions, unbounded retention, resize pressure, and mistaken snapshot assumptions.

Production Failure Patterns

Frequent failures include an unbounded map used as a cache, a slow computeIfAbsent loader that concentrates requests on hot keys, mutable values changed unsafely, and an assumed global invariant built from size plus put. A hash collision hotspot can produce bin contention. Metrics code can also mistake weakly consistent totals for exact accounting.

Diagnose key cardinality, retained value size, hot-key distribution, mapping-function latency, collision behavior, resize allocation, blocked threads, and the thread safety of value objects. A concurrency-safe container is only one layer of a concurrency-safe design.

Interview-Ready Summary

Modern ConcurrentHashMap uses a bucket table with CAS, bin-level coordination, tree bins, and cooperative resizing rather than the old fixed-segment architecture. Reads are generally non-blocking and completed updates safely publish their mappings.

Use putIfAbsent, compute, merge, replace, and conditional remove for atomic map operations. Nulls are forbidden so null lookup has clear absence meaning. Iterators are weakly consistent rather than fail-fast and do not form snapshots.

Atomicity is normally scoped to a mapping, not multiple keys or mutable objects stored as values. Long-running computations, unbounded retention, hot keys, and downstream side effects remain application concerns.

Revision Notes

- ConcurrentHashMap is not a HashMap protected by one global lock.
- The old segment explanation and concurrencyLevel-as-segment-count model are outdated.
- Reads generally avoid bin locks; empty-bin insertion can use compare-and-set.
- Populated bins coordinate locally, so poor hashes and hot keys reduce concurrency.
- Completed updates safely publish references; later value mutation needs separate synchronization.
- Prefer atomic APIs over manual check-then-act sequences.
- `computeIfAbsent` is per-map atomicity, not a distributed transaction.
- Java 21 invokes its function once per absent-key invocation, not once forever.
- Keep remapping functions short, non-recursive, and controlled in their side effects.
- Null keys and values are forbidden.
- Aggregate size under mutation is a moving observation backed by striped counts.
- During cooperative resize, old and new tables coexist and forwarding nodes direct operations.
- Weakly consistent traversal is not a snapshot; establish a boundary when exactness matters.
- LongAdder suits high-contention approximate statistics.
- Per-key atomicity does not enforce cross-key invariants.
- Thread-safe maps can contain thread-unsafe values.
- Unbounded ConcurrentHashMap caches still leak by retention policy.
- Table initialization is coordinated and may be deferred until first use.
- Diagnose hot keys, loader latency, mutable values, resizing, cardinality, and snapshot assumptions.